المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⵓⵙⵙⵏⵉⵡⵉⵏ ⵜⵉⵏⵎⵍⴰⵏ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

COMPTE RENDU DU TP-2

1ÈRE ANNÉE – FILIÈRE INTELLIGENCE ARTIFICIELLE ET CYBERSÉCURITÉ

MODULE : ARCHITECTURE LOGICIELLE

Réalisé par :

ASIM ZINEB

Encadré par :

Pr. BE ELBAGHAZAOUI

Année unniversitaire:2025_2026

Objectifs

Ce TP consiste à Réorganiser l'application monolithique en modules métier bien séparés, en définissant des limites claires entre les domaines, en mettant en place des interfaces de communication, et en appliquant les bonnes pratiques d'architecture (DTO, Mapper, Service Layer) afin de faciliter une future migration vers une architecture microservices.

Outils utilisés

Le TP2 repose sur le projet réalisé lors du TP1, qui sert de point de départ. La principale nouveauté est l'ajout de MapStruct, un framework de mapping permettant de générer automatiquement, à la compilation, le code de conversion entre les entités et les DTO. Son utilisation implique une mise à jour du fichier *pom.xml*, notamment pour intégrer la dépendance correspondante et configurer le plugin Maven Compiler. Les autres outils utilisés restent les mêmes que pour le TP1, à savoir JDK 17, Maven, IntelliJ IDEA, PostgreSQL et Postman.

Réalisation du TP

1. Implémentation du module Product

- **Créer les DTOs**
  - **ProductDTO.java**

```java
package com.ecommerce.monolith.product.dto;
import ...
@Data  33 usages
public class ProductDTO {
    private Long id;
    private String name;
    private String description;
    private BigDecimal price;
    private Integer stock;
    private Long categoryId;
    private String categoryName;
}
```

- **CreateProductRequest.java**

```java
package com.ecommerce.monolith.product.dto;
import ...
@Data  15 usages
public class CreateProductRequest {
    @NotBlank(message = "Product name is required")
    private String name;
    private String description;
    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    private BigDecimal price;
    @PositiveOrZero(message = "Stock cannot be negative")
    private Integer stock = 0;
    @NotNull(message = "Category is required")
    private Long categoryId;
}
```

- **Création de Mapper**

```java
package com.ecommerce.monolith.product.mapper;
import ...
@Mapper(componentModel = "spring")  3 usages  1 implementation
public interface ProductMapper {
    @Mapping(target = "categoryId", source = "category.id")  4 usages  1 implementation
    @Mapping(target = "categoryName", source = "category.name")
    ProductDTO toDTO(Product product);
    List<ProductDTO> toDTOList(List<Product> products);  1 usage  1 implementation
    @Mapping(target = "id", ignore = true)  1 usage  1 implementation
    @Mapping(target = "category", ignore = true)
    Product toEntity(CreateProductRequest request);
    @Mapping(target = "id", ignore = true)  1 usage  1 implementation
    @Mapping(target = "category", ignore = true)
    void updateEntity(CreateProductRequest request, @MappingTarget Product product);
}
```

Refactorisation du service

- **ProductService.java**

```java
package com.ecommerce.monolith.product.service;
import ...
public interface ProductService {  3 usages  1 implementation
    List<ProductDTO> getAllProducts();  1 usage  1 implementation

    ProductDTO getProductById(Long id);  2 usages  1 implementation

    ProductDTO createProduct(CreateProductRequest request);  1 usage  1 implementation

    ProductDTO updateProduct(Long id, CreateProductRequest request);  1 usage  1 implementation

    void deleteProduct(Long id);  1 usage  1 implementation
}
```

- **ProductServiceImpl.java**

```java
package com.ecommerce.monolith.product.service;
import ...
@Service
@RequiredArgsConstructor
@Transactional
public class ProductServiceImpl implements ProductService, ProductPublicService {
    private final ProductRepository repository;
    private final CategoryRepository categoryRepository;
    private final ProductMapper mapper;
    @Override  1 usage
    @Transactional(readOnly = true)
    public List<ProductDTO> getAllProducts() { return mapper.toDTOList(repository.findAll()); }
    @Override  2 usages
    @Transactional(readOnly = true)
    public ProductDTO getProductById(Long id) { return mapper.toDTO(findProduct(id)); }
    @Override  1 usage
    public ProductDTO createProduct(CreateProductRequest request) {
        Product product = mapper.toEntity(request);
        product.setCategory(findCategory(request.getCategoryId()));
        Product saved = repository.save(product);
        return mapper.toDTO(saved);
    }
    @Override  1 usage
    public ProductDTO updateProduct(Long id, CreateProductRequest request) {
        Product product = findProduct(id);
        mapper.updateEntity(request, product);
        product.setCategory(findCategory(request.getCategoryId()));
        Product updated = repository.save(product);
        return mapper.toDTO(updated);
    }
```

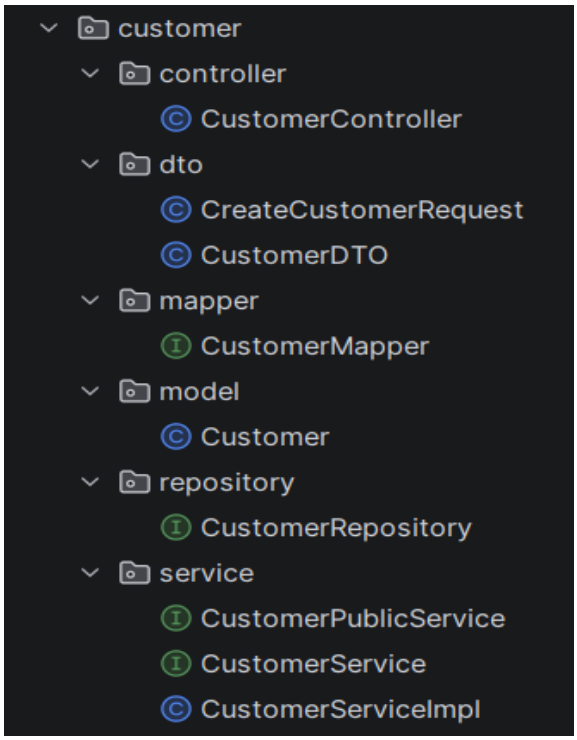
ProductPublicService expose uniquement les opérations accessibles aux autres modules. ProductService étend cette interface en ajoutant le CRUD complet pour usage interne. ProductServiceImpl est l'implémentation concrète des deux, contenant toute la logique métier.

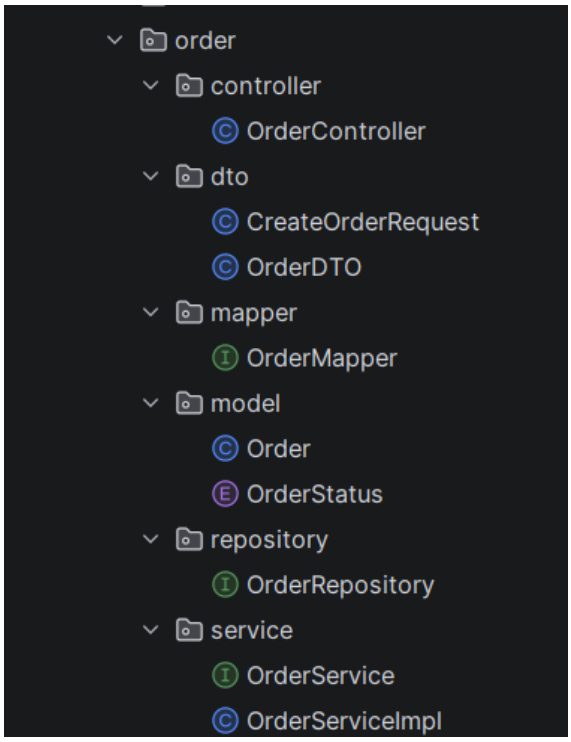
Adaptation du Controller

```java
package com.ecommerce.monolith.product.controller;
import ...
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
@Validated
public class ProductController {
    private final ProductService productService;
    @GetMapping
    public ResponseEntity<List<ProductDTO>> getAll() { return ResponseEntity.ok(productService.getAllProducts()); }
    @GetMapping("/{id}")
    public ResponseEntity<ProductDTO> getById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.getProductById(id));
    }
    @PostMapping
    public ResponseEntity<ProductDTO> create(@Valid @RequestBody CreateProductRequest request) {
        ProductDTO created = productService.createProduct(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }
    @PutMapping("/{id}")
    public ResponseEntity<ProductDTO> update(@PathVariable Long id, @Valid @RequestBody CreateProductRequest request) {
        return ResponseEntity.ok(productService.updateProduct(id, request));
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}
```

2. Implémentation des modules Customer et order

Le module **Customer** et le module **Order** ont été implémentés en suivant exactement la même architecture que le module Product, comprenant l'entité, le repository, les DTOs, le mapper, ainsi que l'interface service et son implémentation, tout en assurant une cohérence globale de l'application.





- ✓ **Méthode de vérification l'existence du Customer**

La méthode existsById() est définie dans CustomerPublicService, déclarée dans CustomerService et implémentée dans CustomerServiceImpl. Elle est réutilisée dans le module Order dans deux contextes : lors de la création d'une commande et lors de la récupération de l'historique, afin de vérifier l'existence du client avant toute opération.

CustomerServiceImpl.java

```java
package com.ecommerce.monolith.customer.service;
import ...
@Service
@RequiredArgsConstructor
@Transactional
public class CustomerServiceImpl implements CustomerService, CustomerPublicService {
    private final CustomerRepository repository;
    private final CustomerMapper mapper;
    @Override  1 usage
    @Transactional(readOnly = true)
    public List<CustomerDTO> getAllCustomers() { return mapper.toDTOList(repository.findAll()); }
    @Override  2 usages
    @Transactional(readOnly = true)
    public CustomerDTO getCustomerById(Long id) { return mapper.toDTO(findCustomer(id)); }
    @Override  1 usage
    public CustomerDTO createCustomer(CreateCustomerRequest request) {
        Customer customer = mapper.toEntity(request);
        return mapper.toDTO(repository.save(customer));
    }
    @Override  1 usage
    public CustomerDTO updateCustomer(Long id, CreateCustomerRequest request) {
        Customer customer = findCustomer(id);
        mapper.updateEntity(request, customer);
        return mapper.toDTO(repository.save(customer));
    }
    @Override  1 usage
    public void deleteCustomer(Long id) {
        if (!repository.existsById(id)) {
            throw new IllegalArgumentException("Customer not found with id: " + id);
        }
        @Override  1 usage
        @Transactional(readOnly = true)
        public boolean customerExists(Long id) { return repository.existsById(id); }

        @Override  3 usages
        @Transactional(readOnly = true)
        public CustomerDTO requireCustomer(Long id) { return getCustomerById(id); }

        private Customer findCustomer(Long id) {  2 usages
            return repository.findById(id)
                    .orElseThrow(() -> new IllegalArgumentException("Customer not found with id: " + id));
        }
    }
```

CustomerPublicService.java

```java
package com.ecommerce.monolith.customer.service;

import com.ecommerce.monolith.customer.dto.CustomerDTO;

public interface CustomerPublicService {  3 usages  1 implementation
    boolean customerExists(Long id);  no usages  1 implementation
    CustomerDTO requireCustomer(Long id);  3 usages  1 implementation
}
```

CustomerService.java

```java
package com.ecommerce.monolith.customer.service;
import ...
public interface CustomerService {  3 usages  1 implementation
    List<CustomerDTO> getAllCustomers();  1 usage  1 implementation
    CustomerDTO getCustomerById(Long id);  2 usages  1 implementation
    boolean customerExists(Long id);  1 usage  1 implementation
    CustomerDTO createCustomer(CreateCustomerRequest request);  1 usage  1 implementation
    CustomerDTO updateCustomer(Long id, CreateCustomerRequest request);  1 usage  1 implementation
    void deleteCustomer(Long id);  1 usage  1 implementation
}
```

Ainsi, l'endpoint getOrdersByCustomerId() est défini dans OrderService, implémenté dans OrderServiceImpl et exposé via OrderController. Il est accessible via GET /api/orders/customer/{customerId} et retourne l'historique des commandes d'un client après avoir vérifié son existence via existsById().

OrderController.java

```java
package com.ecommerce.monolith.customer.controller;
import ...
@RestController
@RequestMapping("/api/customers")
@RequiredArgsConstructor
public class CustomerController {
    private final CustomerService customerService;
    @GetMapping
    public ResponseEntity<List<CustomerDTO>> getAll() { return ResponseEntity.ok(customerService.getAllCustomers()); }
    @GetMapping("/{id}")
    public ResponseEntity<CustomerDTO> getById(@PathVariable Long id) {
        return ResponseEntity.ok(customerService.getCustomerById(id));
    }
    @GetMapping("/{id}/exists")
    public ResponseEntity<Boolean> exists(@PathVariable Long id) {
        return ResponseEntity.ok(customerService.customerExists(id));
    }
    @PostMapping
    public ResponseEntity<CustomerDTO> create(@Valid @RequestBody CreateCustomerRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED).body(customerService.createCustomer(request));
    }
    @PutMapping("/{id}")
    public ResponseEntity<CustomerDTO> update(@PathVariable Long id, @Valid @RequestBody CreateCustomerRequest request) {
        return ResponseEntity.ok(customerService.updateCustomer(id, request));
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        customerService.deleteCustomer(id);
        return ResponseEntity.noContent().build();
    }
```

- ✓ **Gestion globale des erreurs**

La classe ApiExceptionHandler annotée @RestControllerAdvice intercepte toutes les exceptions de l'application et retourne des réponses HTTP cohérentes. Elle gère deux cas : IllegalArgumentException pour les ressources introuvables (404 Not Found) et MethodArgumentNotValidException pour les erreurs de validation (400 Bad Request).

3. Test avec postman

Module Product

- Créer un produit

POST http://localhost:8080/api/products

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
{
    "name": "prod",
    "description": "performant",
    "price": 129.99,
    "stock": 6,
    "categoryId": 1
}
```

Body Cookies Headers (5) Test Results Status: 201 Created Time:

Pretty Raw Preview Visualize JSON

```
{
    "id": 2,
    "name": "prod",
    "description": "performant",
    "price": 129.99,
    "stock": 6,
    "categoryId": 1,
    "categoryName": "Informatique"
}
```

- Lister tous les produits

GET http://localhost:8080/api/products

Params Authorization Headers (7) Body Pre-request Script Tests Settings

Body Cookies Headers (5) Test Results Status: 200 OK Time:

Pretty Raw Preview Visualize JSON

```
[
    {
        "id": 1,
        "name": "Laptop ",
        "description": "Ordinateur portable ",
        "price": 1299.99,
        "stock": 10,
        "categoryId": 1,
        "categoryName": "Informatique"
    },
    {
        "id": 2,
        "name": "prod",
        "description": "performant",
        "price": 129.99,
        "stock": 6,
        "categoryId": 1,
        "categoryName": "Informatique"
    }
]
```

Module Customer

- Créer un client

POST http://localhost:8080/api/customers

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
{
    "firstName": "ali",
    "lastName": "B",
    "email": "ali.b@gmail.com",
    "phone": "0612345678"
}
```

Body Cookies Headers (5) Test Results Status: 201 Created

Pretty Raw Preview Visualize JSON

```
{
    "id": 5,
    "firstName": "ali",
    "lastName": "B",
    "email": "ali.b@gmail.com"
}
```

- Récupérer un client

GET http://localhost:8080/api/customers

Params Authorization Headers (7) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary

This request does not have a body

Body Cookies Headers (5) Test Results Status: 200 OK

Pretty Raw Preview Visualize JSON

```
    },
    {
        "id": 2,
        "firstName": "John",
        "lastName": "Doe",
        "email": "john@example.com"
    },
    {
        "id": 4,
        "firstName": "zineb",
        "lastName": "B",
        "email": "zineb.bli@gmail.com"
    },
    {
        "id": 5,
        "firstName": "ali",
        "lastName": "B",
        "email": "ali.b@gmail.com"
    }
]
```

Module Order

- Créer une commande

POST http://localhost:8080/api/orders

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
{ "customerId": 1, "productId": 1, "quantity": 2}
```

Body Cookies Headers (5) Test Results Status: 201 Created

Pretty Raw Preview Visualize JSON


```
{
    "id": 2,
    "customerId": 1,
    "customerName": "zineb B",
    "productId": 1,
    "productName": "Laptop ",
    "quantity": 2,
    "unitPrice": 1299.99,
    "totalPrice": 2599.98,
    "status": "CREATED",
    "createdAt": "2026-03-20T23:20:14.2676754"
}
```


- Historique des commandes d'un client

GET http://localhost:8080/api/orders/customer/1

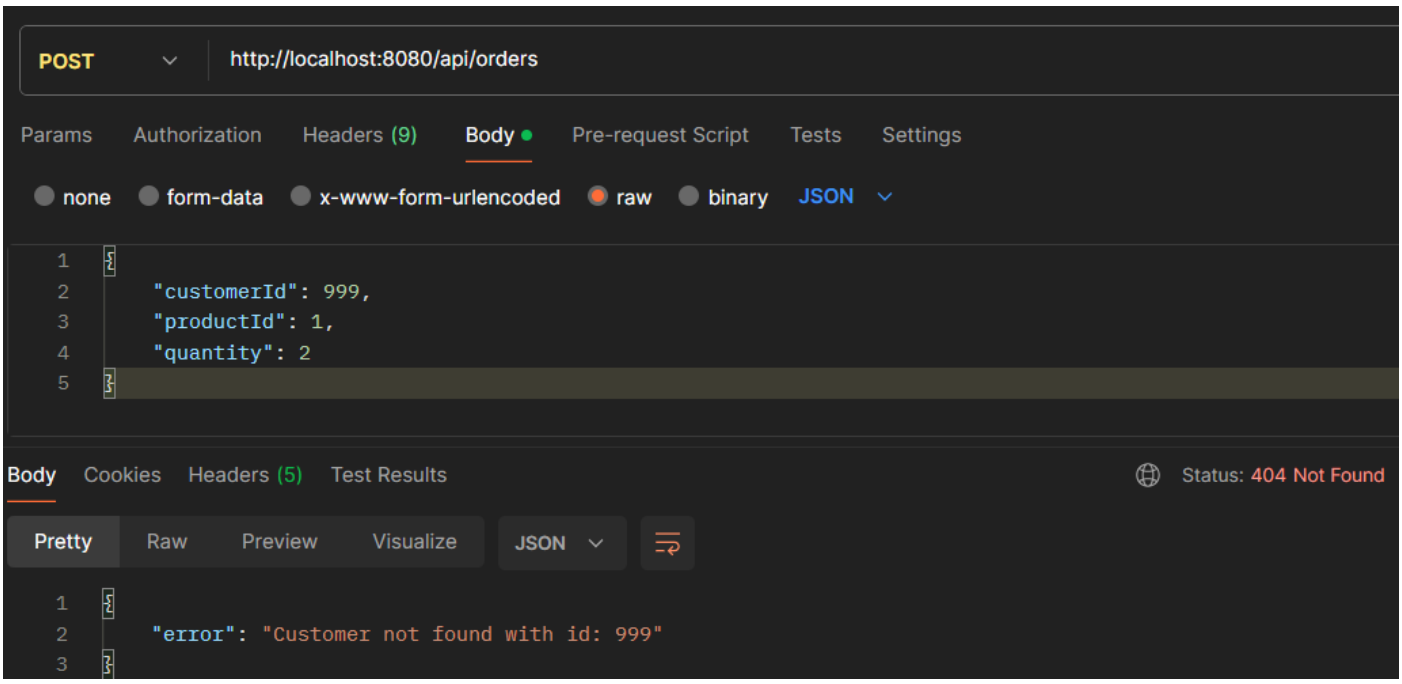
Params Authorization Headers (7) Body Pre-request Script Tests Settings

Body Cookies Headers (5) Test Results Status: 200 OK

Pretty Raw Preview Visualize JSON


```
[
    {
        "id": 2,
        "customerId": 1,
        "customerName": "zineb B",
        "productId": 1,
        "productName": "Laptop ",
        "quantity": 2,
        "unitPrice": 1299.99,
        "totalPrice": 2599.98,
        "status": "CREATED",
        "createdAt": "2026-03-20T23:20:14.267675"
    },
    {
        "id": 1,
        "customerId": 1,
        "customerName": "zineb B",
        "productId": 1,
        "productName": "Laptop ",
        "quantity": 2,
        "unitPrice": 1299.99,
        "totalPrice": 2599.98,
        "status": "CREATED",
        "createdAt": "2026-03-20T20:45:38.718641"
```

- commande avec un customerId inexistant, retourne erreur "Customer not found"



Conclusion

L'application a été restructurée en modules Product, Customer et Order, avec DTOs et services pour séparer présentation et persistance. Le couplage entre modules est faible, le module Order utilisant uniquement les services des autres modules. Cette organisation facilite la maintenabilité et prépare une éventuelle migration vers une architecture microservices.